# ESTRUCTURAS DE DATOS I

**Tema 0: Introducción al Diseño Modular**

# ÍNDICE

1. Programación Modular.
2. Concepto de Módulo.
3. Creación y Utilización de Módulos.
4. Importación y Exportación.
5. Definición y Uso de Módulos.

# 1. Programación Modular

- Programación Estructurada: programa → 1 a n ***funciones***.
- ***IDEA:***

  *¿Cómo sería posible utilizar definiciones y funciones de un programa en otros sin tener que rescribirlas?*

- Programación Modular: programa → 1 a n **módulos.**
- Ventajas:
  - Disminuye la complejidad del problema.
  - Permite la compilación separada.
  - Programas más fáciles de modificar.
  - Permite reutilizar código ya probado.

# 1. Concepto de Módulo

- **Módulo** (o biblioteca)**:**

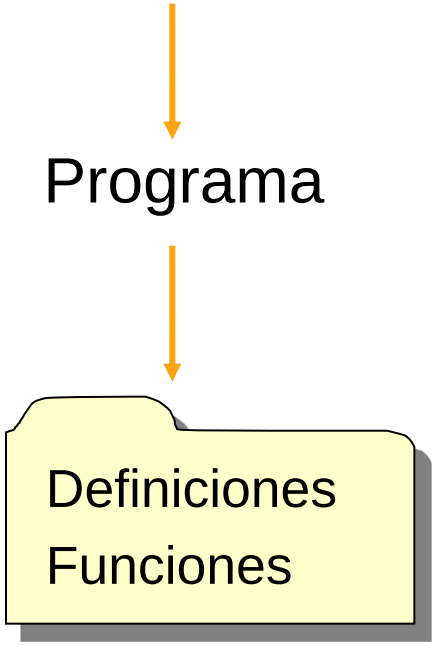
  Colección de declaraciones (constantes, tipos, clases, funciones y/o métodos) definidas en un ámbito de visibilidad cerrada.

- **Diseño Modular:**

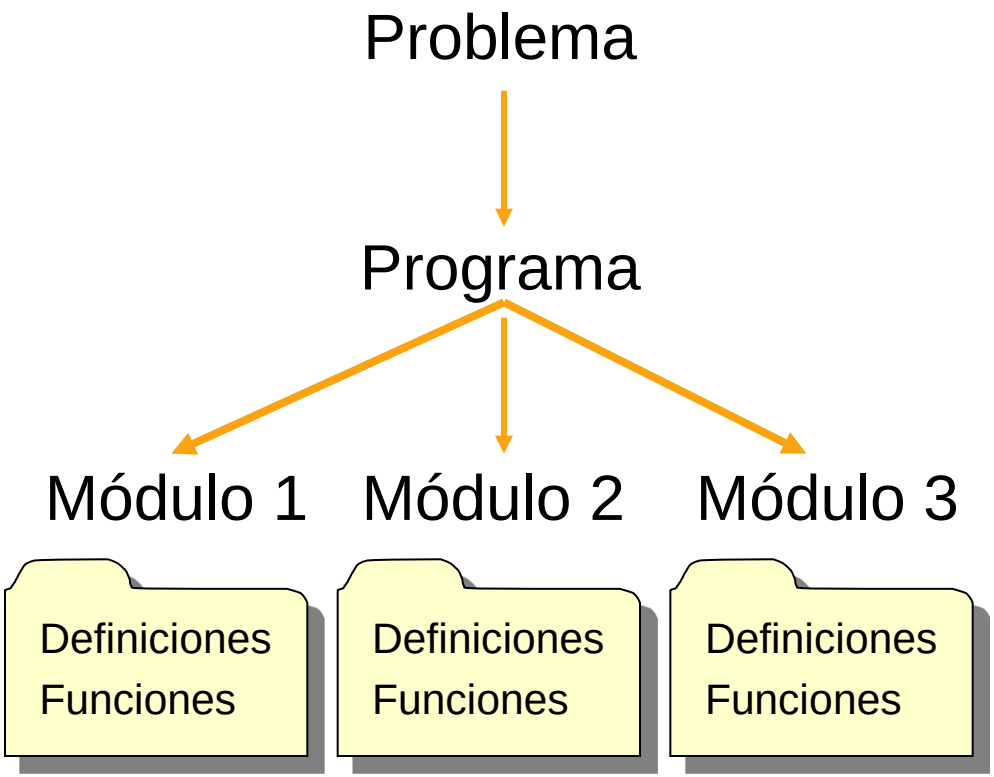
  Descomposición de un problema en subproblemas resueltos en unidades independientes (módulos o bibliotecas). Conectados convenientemente estos módulos, se resolverá el problema**.**

# *Diseño Descendente vs Diseño Modular*

## Diseño Descendente




## Diseño Modular



# 3. Creación y Utilización de Módulos

## Usuario

- Uso para aplicación concreta.
- Sólo conoce la especificación no su implementación.
- En el main() o en otro módulo.
- Incluye archivo cabecera.
- En C++ se genera un proyecto para unir los módulos con el programa principal.

## Creador

- Creación  ante necesidad.
- Módulo general y reutilizable.
- Implementa los elementos.
- Documentación de su definición.
- Proteger estructura de la información.

# 4. Importación y Exportación

## Importación

- Un programa importará definiciones de otros módulos.
- Los módulos pueden importar de otros módulos.
- El compilador proporciona un conjunto de módulos que se pueden importar también.
- Se realiza con la directiva de compilación `#include`.

## Exportación

- Los módulos se usan para exportar recursos a otros módulos o programas.
- Se exportan constantes, tipos, clases, variables y/o funciones.
- Todo lo que se exporta estará definido en un fichero de definiciones no ejecutable.

# Partes de un Módulo en C++

## Especificación (Definición, cabecera o Interfaz)

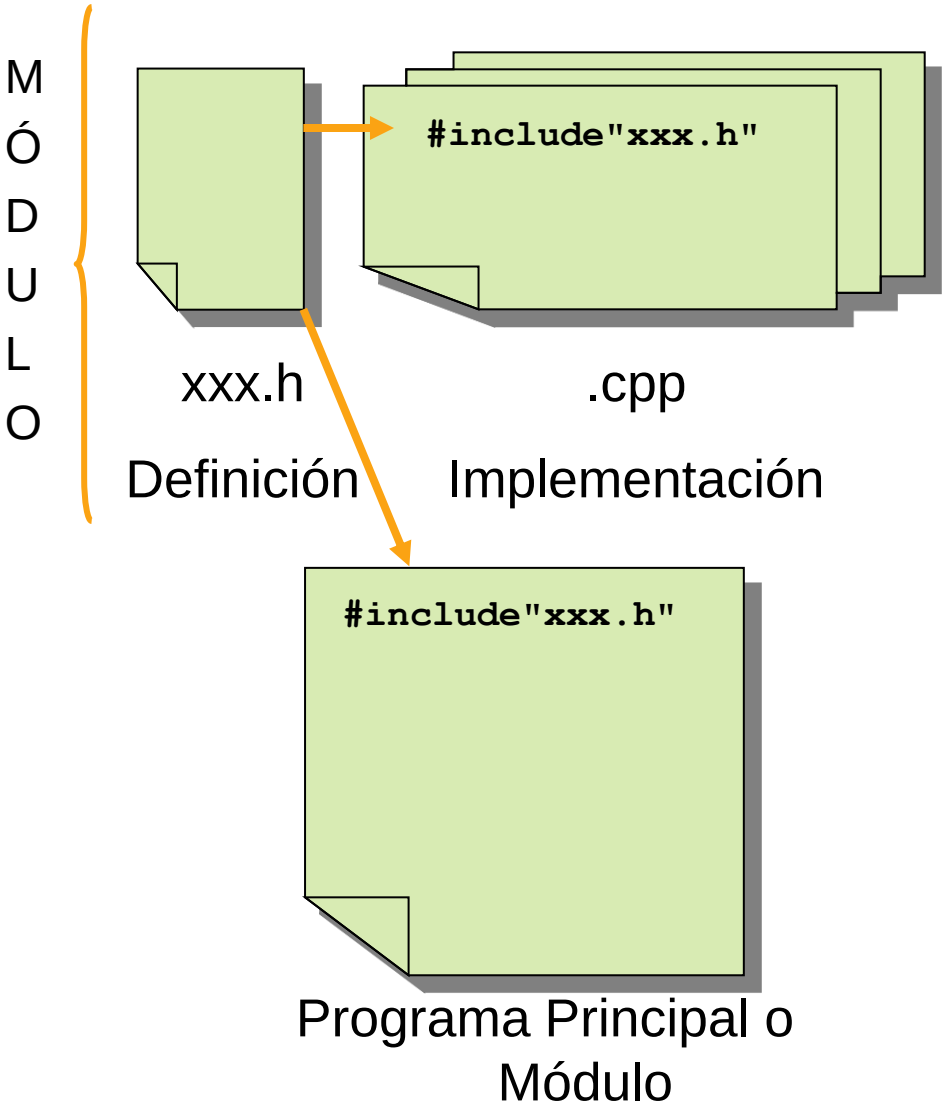
- Interfaz del módulo con el usuario del módulo.
- Definiciones que el módulo exporta:
  - Constantes
  - Tipos / Funciones / Métodos
  - Clases
  - Etc,…
- Normalmente ficheros con extensión en .h.

## Implementación (Cuerpo del módulo)

- Implementación de las definiciones anteriores.
- Uno o varios ficheros .cpp.
- Debe importar el fichero de definición:

```
#include <xxx>
#include <xxx.h> ó
#include "xxx.h"
```

# Partes de un Módulo en C++




## Compilación Separada

- Se compilan los ficheros fuentes para obtener los f. objeto y se enlazan para obtener el programa ejecutable →PROYECTO C++
- Un cambio en el p.p. implica sólo recompilarlo y volver a enlazar el programa entero.
- Un cambio en la definición, implica recompilar su implementación y los módulos que importen algo de dicha biblioteca.

# Uso de directivas de compilación

- Algunas directivas de compilación, además de #include, usadas en Programación modular son:

```
#ifndef _NombreFichero_h_
#define _NombreFichero_h_
…
#endif
```

*"Si no esta definida la "macro"* `_NombreFichero_h_` *defínala. En caso contrario no hacer nada ya que no hay nada después de la directiva* `#endif`*"*

- Se utiliza para impedir que se produzcan errores de compilación cuando un fichero de definición es importado dos veces por el mismo módulo.

# Utilidad de las Directivas de Compilación

**Ejemplo**: Realizamos un proyecto y tenemos:

```
class Ventana {
  ...; // atributos
  ...; // prototipos de métodos
 }
```

Fichero ventana.h

```
#include "ventana.h"
class Coche {
    //hace uso de ventanas
   ...; // atributos
    ...; // prototipos de métodos
}
```

Fichero coche.h

```
#include "ventanas.h"
#include "coche.h"
….
```

Fichero main.cpp

¿qué pasaría?

En el contexto del módulo main (main.cpp) la clase ventana está "redefinida" y daría error. Una forma de solucionarlo es darse cuenta que incorporar coche.h incorpora ventana.h. La segunda ,y es la que proponemos, es usar directivas de compilación:

```
  #ifndef VENTANADEF
  #define VENTADEF
class Ventana {
    ...; // atributos
    ...; // prototipos de
métodos
 }
```

**fichero Ventana.h**

# Utilidad de las Directivas de Compilación

Fichero ventana.h

```
#ifndef VENTANADEF
#define VENTANADEF

  class Ventana {
  ...; // atributos
  ...; // prototipos de métodos
  }

#endif
```

```
#include "VENTANA.h"
#include "COCHE.h"
int main() {
   ...;
   Ventana V;
   Coche c;
   ...;
}
```

Fichero coche.h

```
#ifndef COCHEDEF
#define COCHEDEF
   #include "ventana.h"
    class Coche{
    ...; // atributos
    ...; // prototipos de métodos
    }

#endif
```

# 5. Definición y Uso de Módulos.

```
#ifndef _Ccomplejo_h
#define _Ccomplejo_h
class complejo {
    double real, imag;
public:
    complejo(void) { real = 0.0; imag = 0.0; }
    complejo(double re, double im=0.0) { real = re; imag = im;}
    complejo(const complejo& r) { real = r.real; imag = r.imag;}
    void SetReal(double re) { real = re; }
    void SetImag(double im) { imag = im; }
    double GetReal(void) {return real; }
    double GetImag(void) {return imag; }
    complejo operator+(int e);
    complejo operator+(complejo c);     //por valor
    complejo operator-(const complejo& c); //por ref
    int operator== (complejo c);
    complejo operator=(complejo c);
    complejo operator++(); // ++obj;
    complejo operator++(int notused); //obj++
    complejo operator-();
    void mostrar();
};

#endif
```

Fichero de Definición de la clase complejo:

Ccomplejo.h

```
#include "Ccomplejo.h"
#include <iostream>
using namespace std;
// sobrecarga de + para un entero
complejo complejo::operator+ (int e) {
 complejo suma;
 suma.real = real + e;   suma.imag = imag;
 return suma;
}
// sobrecarga de +
complejo complejo::operator+ (complejo c) {
 complejo suma;
 suma.real = real + c.real;
 suma.imag = imag + c.imag;
 return suma;
}
// sobrecarga de -
complejo complejo::operator- (const complejo &c) {
 complejo resta;
 resta.real = real - c.real;
 resta.imag = imag - c.imag;
 return resta;
}
```

Fichero de Implementación de la clase complejo:

Ccomplejo.cpp

```
// sobrecarga de ==
int complejo::operator== (complejo c) {
 if (real==c.real && imag==c.imag)
  return 1;
 else
  return 0;
}
// sobrecarga de =
complejo complejo::operator= (complejo c) {
 real=c.real; imag=c.imag;
 return c;
}
// sobrecarga de - unario
complejo complejo::operator-() {
complejo copia;
copia.real=-real;
copia.imag=-imag;
return copia;
}
```

```
// sobrecarga de ++ (++obj)
complejo complejo::operator++() {
complejo copia;
real++;
copia.real=real;
copia.imag=imag;
return copia;
}
// sobrecarga de ++ (obj++)
complejo complejo::operator++(int notused){
complejo copia;
copia.real=real;
copia.imag=imag;
real++;
return copia;
}
// mostrar complejo
void complejo::mostrar(){
if (imag>0)
    cout<<real<<"+"<<imag<<"i"<<'\n';
else
    cout<<real<<imag<<"i"<<'\n';
}
 //fin del módulo
```

Fichero de Implementación de la clase complejo: Ccomplejo.cpp

```
#include <iostream>
#include "Ccomplejo.h"
using namespace std;
int main()
{
complejo c1(2,3), c2(-1,4),c3;
c3=c1+c2;c3.mostrar();
c3=c1-c2;c3.mostrar();
c3=c1*c2;c3.mostrar();
c1=++c2;
c2=-c3;
if (c1==c2) c3=c1;
else c3=c2;
c3.mostrar();
c3=c2+2;
c3.mostrar();
system ("pause");
return 0;
}
```

La salida del programa será:

```
1+7i
3-1i
-3+1i
-1+1i
Presione una tecla para continuar...
```

Programa que hace uso del módulo